SYSTEMS PROGRAMMING · MEMORY MODEL

How C Manages Memory

A plain-language guide to C's memory model, and the flaws that come with it.

PART 1 · FOUNDATIONS

C gives you one of the simplest possible pictures of a computer's memory: a long row of bytes, and pointers that are just numbers pointing into that row. That simplicity is why C is fast, small, and still the language of operating systems, drivers, and embedded devices. It is also why C programs are famous for memory bugs that are silent, random, and sometimes dangerous. This guide explains how C's memory model works, then walks through the flaws one by one, in plain language and with pictures.

WHAT IS INSIDE

01 The one big idea

02 Where things live

03 Three lifetimes

04 The stack

05 The heap

06 Pointers

07 Aliasing

08 Undefined behavior

09 The catalog of flaws

10 Why C is built this way

11 What people do about it

12 Summary

01 The one big idea

C's memory model starts with a single idea: **memory is one long array of bytes.** Every byte has an address, and addresses are just integers. A variable is a name for some bytes. An array is a row of bytes. A struct is bytes arranged in a particular order. A pointer is a number that names an address.

That is the whole model. There is no hidden object table, no garbage collector, no per-object metadata that the language tracks. A pointer does not know how big the thing it points to is, or whether that thing is still alive. It is just a number.

This is the source of both C's power and its danger. Because a pointer is just a number, C can talk to hardware registers, memory-mapped devices, and memory shared with other languages. Because a pointer is just a number, it can also point at garbage.

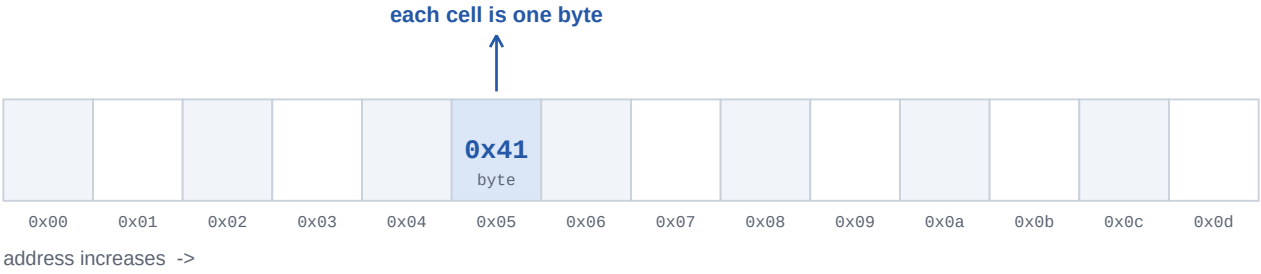

Figure 1. C's model of memory. A pointer is just an index into a flat array of bytes.

> **The rule that is not enforced.** C says a pointer may only point inside a live object. Nothing in the language or the machine checks that. The rule is a convention between you and the compiler.

One important detail: the C standard describes an **abstract machine**, not your actual CPU. The compiler can keep variables in registers, reorder operations, and delete code, as long as the observable behavior of a correct program stays the same. When your program breaks the rules, the compiler is allowed to assume it did not. That is where undefined behavior comes from, and we return to it in Section 8.

02 Where things live

In practice, a running C program's address space is split into a few regions. The picture below is the conventional layout on Linux, Windows and macOS. The C standard does not require it, and embedded systems often look different, but the idea is the same everywhere.

- **Text** — the machine code of your program. Usually read-only.
- **Data** — global and static variables that you initialized with a value.
- **BSS** — global and static variables that start at zero.
- **Heap** — memory you request at run time with `malloc`. It grows upward.
- **Stack** — memory for function calls and local variables. It grows downward, toward the heap.

The empty space between the heap and the stack is free memory. The heap grows up into it as you allocate, and the stack grows down into it as you call deeper. If they meet, you are out of memory, or more likely you have recursed too far.

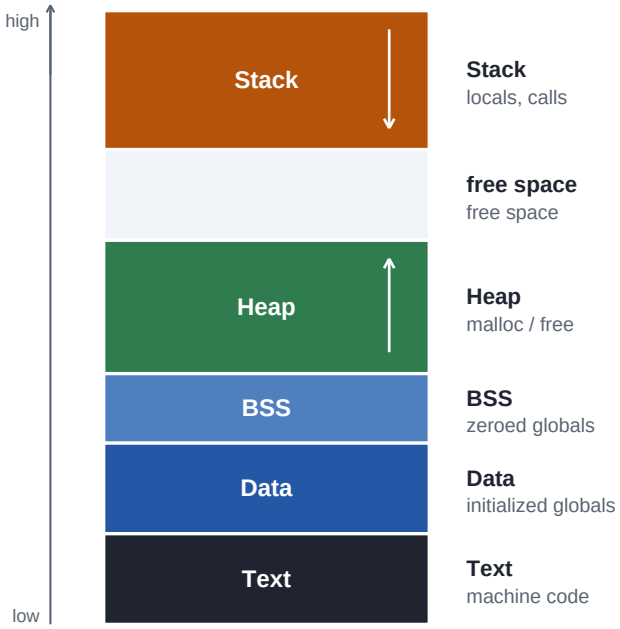

Figure 2. The conventional address space. The heap grows up, the stack grows down, and the gap between them is free memory.

03 Three lifetimes

C groups memory by how long it lives. Three cases cover almost all code, and they answer the question “how long does this memory live?”

Duration	Where	Created	Destroyed	Typical use
Automatic	stack	function is entered	function returns	local variables
Static	data / BSS	program starts	program exits	globals and `static` locals
Dynamic	heap	`malloc()`	`free()`	anything you allocate

(The C standard also defines **thread** storage duration, which behaves like static storage but gives each thread its own copy, and it calls heap memory **allocated** storage. We ignore those details here.)

Automatic memory is the easiest: the compiler manages it, and it costs almost nothing. Static memory is nearly as easy: it exists for the whole run. Dynamic memory is the hard one, because **you** are the manager. Every `malloc` needs a matching `free`, at the right time, exactly once. Getting that right in a large program is genuinely difficult, and that is where most C memory bugs come from.

04 The stack

Every time you call a function, the CPU pushes a **stack frame**. The frame holds the arguments, the return address (where to jump back to), saved registers, and the function's local variables. When the function returns, the frame is popped and the stack pointer moves back.

The stack is fast because it is just a pointer bump: allocate by subtracting, free by adding. It is also safe in one specific sense: you cannot leak stack memory, because it is reclaimed automatically on return. But the stack has sharp edges:

- **No bounds checks.** A local array can be written past its end, quietly overwriting the return address or a neighbor.

- **Stack overflow.** Deep recursion or a huge local array runs off the end of the stack and crashes, or worse, on systems without a guard page.
- **Dangling pointers to locals.** If you return a pointer to a local variable, the frame disappears when the function returns. The pointer still holds the old address, which the next call will soon reuse.

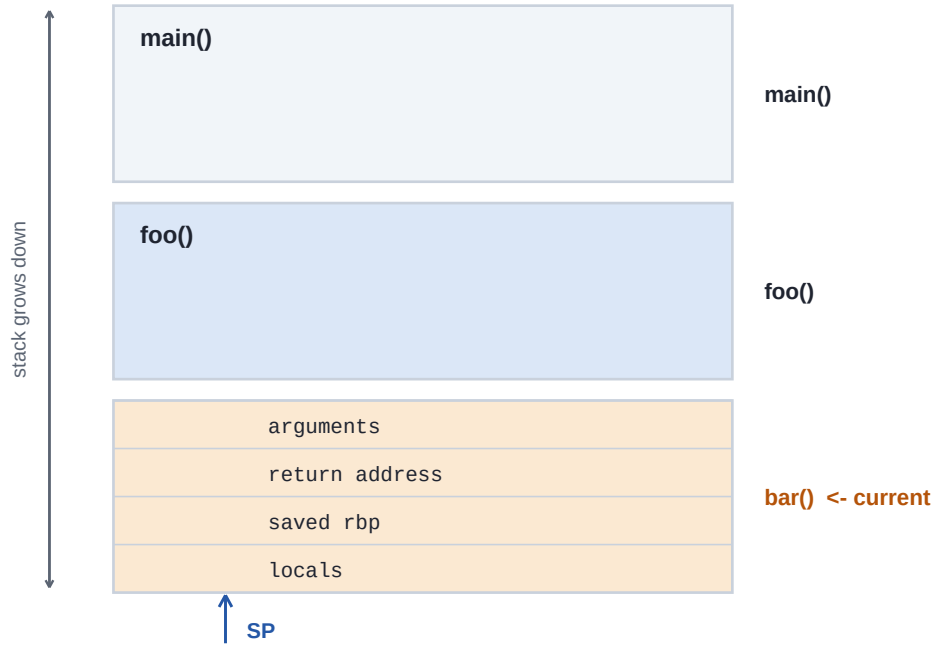

Figure 3. Stack frames. Each call pushes a new frame; returning pops it. The current function's frame is at the lowest address.

05 The heap

The heap is a large pool of memory that the program manages itself. `malloc(n)` asks the allocator for at least *n* bytes and returns a pointer. `free(p)` gives the block back.

Behind the scenes, the allocator keeps **metadata**, usually a small header just before the pointer it returns, recording the block's size and whether it is free. It also keeps a **free list** of available blocks so it can reuse them. When you call `free`, the block goes back on that list, and a later `malloc` may hand the same bytes to someone else.

This design is fast and flexible, but every part of it trusts the program:

- **Memory leaks.** Forget to free, and the block stays reserved forever.
- **Use-after-free.** Keep using a pointer after free, and you are reading or writing memory that now belongs to someone else.
- **Double free.** Free the same pointer twice, and the allocator's free list gets corrupted.
- **Fragmentation.** After many alloc/free cycles, free memory is split into small scattered pieces, so a large request fails even though enough total memory exists.
- **Metadata corruption.** A buffer overflow can overwrite the hidden header in front of the next block. Attackers use this to make the allocator write to an address of their choosing.

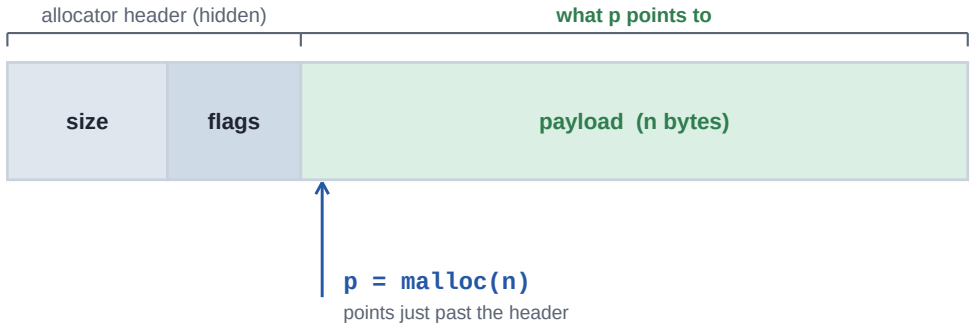

Figure 4. A heap block. The allocator stores hidden metadata just before the address it returns to you.

06 Pointers: address plus type

A pointer has two things: an address and a type. The type matters for arithmetic. If `p` is an `int *`, then `p + 1` moves forward by `sizeof(int)` bytes, usually 4, not by one byte. The compiler multiplies for you.

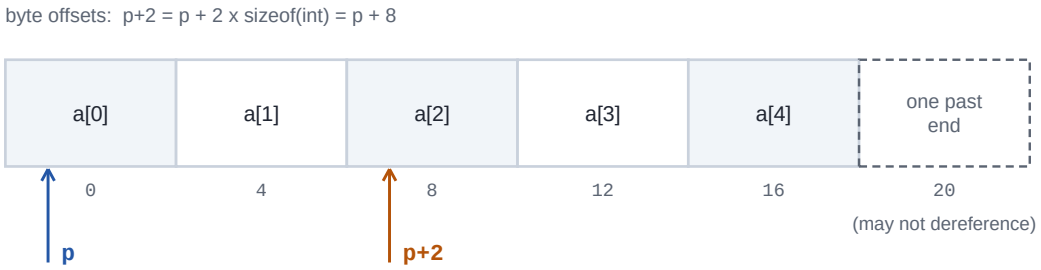

Figure 5. Pointer arithmetic moves in whole objects. A pointer may be formed one past the end, but not dereferenced there.

Pointer arithmetic is only *defined* inside a single array object, plus one position past the end. You may form a pointer to `a[5]` for an array of five elements and compare it, but you may not dereference it. You may not compute a pointer that jumps from one object into another, even if the addresses happen to be adjacent.

Arrays are laid out contiguously: a[0], a[1], and so on, with no gaps. Structs are different. The compiler may insert invisible **padding** between fields so that each field starts at an address the hardware likes (its alignment). So `sizeof(struct)` can be larger than the sum of its fields. This is harmless for correctness, but it surprises people who assume C lays out bytes exactly as written, and it matters when you write a struct to disk or send it over a network.

> Here is the problem: **C does not track which object a pointer came from.** The rule exists in the standard, but nothing in the language or the machine enforces it. You can add a thousand to a pointer and dereference it. You can cast an integer to a pointer. The hardware will happily read whatever bytes are at that address. This is a **provenance** problem: a pointer's right to access an object is not represented anywhere.

07 Aliasing

Two pointers can point at the same object. This is called **aliasing**, and it is completely legal:

```
int  x = 42;
int *p = &x;
int *q = &x;   /* p and q alias: two names for the same bytes */
```

Aliasing is useful. It lets functions take pointers to the same buffer, lets data structures share nodes, and lets you write code that works on “some memory” without copying it.

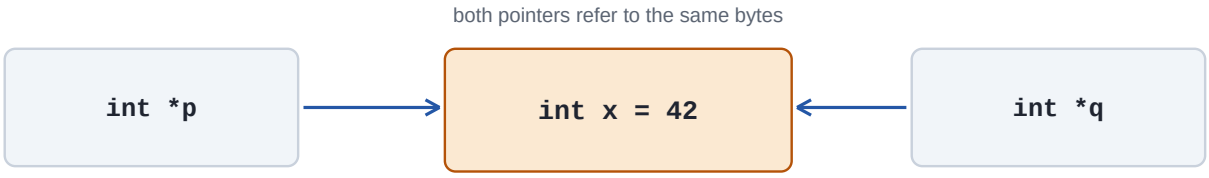

Figure 6. Aliasing. Both pointers reach the same object. This is legal and often useful.

But aliasing makes optimization harder. If the compiler sees `*p = 1;` followed by `x = *q;`, it must assume `*q` might be the memory that `*p` just wrote, so it has to reload. If it knew `p` and `q` never alias, it could keep `x` in a register.

C's answer is `restrict`: a promise from the programmer that a pointer is the *only* way to reach its object for the lifetime of the pointer. The compiler may then optimize aggressively. If you break that promise, the behavior is undefined.

C also has the **strict aliasing rule**: you generally may not access an object through a pointer of an unrelated type. Casting an `int *` to a `float *` and reading through it is undefined even though it compiles. This rule lets the compiler assume that integer stores and floating-point loads do not interfere. It is also a classic source of bugs in code that “just casts”.

08 Undefined behavior

This is the most important idea in the whole document. The C standard defines what a correct program does. For many mistakes it says the behavior is **undefined**, which does not mean “it crashes” or “it does something random”. It means **the standard imposes no requirements at all.**

> **Undefined behavior (UB) is not a crash. It is a license.** If the source contains UB, the compiler may assume it never happens, and it may use that assumption to transform the program.

The result can be code that appears to work in a debug build and breaks in an optimized build; code that works for years and then breaks when the compiler is upgraded; or code where a security check is deleted entirely because the compiler “proved” the check is unnecessary. Common triggers include:

- signed integer overflow
- out-of-bounds access
- dereferencing null or a dangling pointer
- reading an uninitialized variable
- violating strict aliasing
- a data race between two threads

None of these are required to produce a useful error. Many build with no warning and no crash.

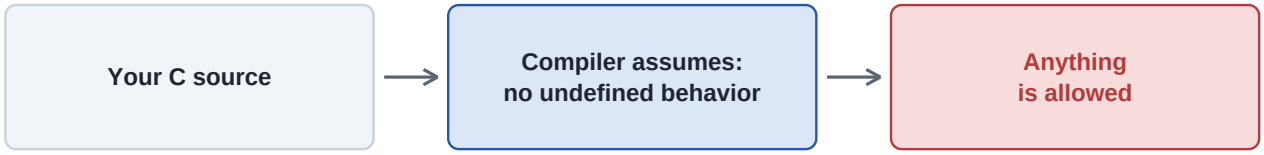

Figure 7. The compiler's contract. Undefined behavior lets the optimizer assume your mistake never happened.

This is the core flaw of C's memory model: **the rules that keep you safe are not enforced, and breaking them does not reliably fail.**

09 The catalog of flaws

Here is the full set, grouped by the kind of rule each one breaks. Every item is a direct consequence of the model we have just described: bytes with addresses, pointers without provenance, and lifetimes managed by hand.

Spatial errors: touching memory you do not own.

- **Buffer overflow / out-of-bounds access.** Writing or reading past the end of an array or allocated block. The bytes there belong to something else: another variable, a return address, or allocator metadata.
- **Stack overflow.** Too much recursion, or local variables larger than the stack, run past the end of the stack region.

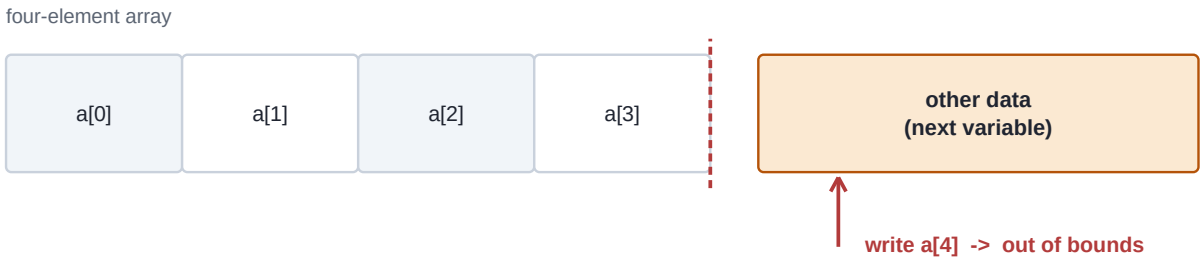

Figure 8. A buffer overflow. Writing a[4] on a four-element array lands in whatever happens to sit next to it.

Temporal errors: using memory at the wrong time.

- **Use-after-free.** The block was returned to the allocator, but a pointer still refers to it. The allocator may have given those bytes to another object, so the old pointer now reads or writes someone else's data.
- **Double free.** The same block is freed twice, which corrupts the allocator's free list and can crash the program or hand an attacker control of a future allocation.
- **Dangling stack pointer.** A function returns a pointer to one of its locals. The frame is gone, but the address remains, and the next call will reuse it.

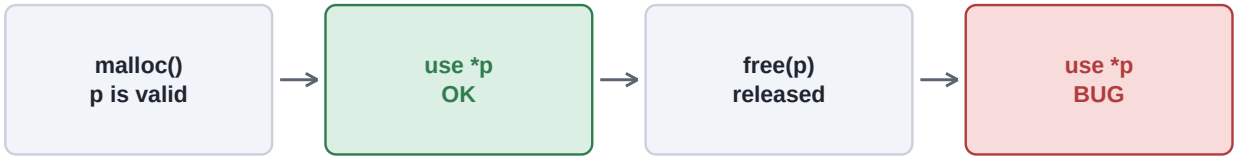

Figure 9. Use-after-free. The pointer outlives the object, and the allocator is free to reuse those bytes.

Type and initialization errors.

- **Type confusion / strict aliasing.** The same bytes are read through a pointer of the wrong type. The compiler may reorder or eliminate accesses because it assumed the types could not overlap.
- **Uninitialized reads.** `malloc` does not zero memory. Reading a value before writing it gives you whatever the previous owner left behind, which is both a wrong-value bug and an information leak.

Concurrency errors.

- **Data race.** Two threads touch the same memory, at least one writes, and there is no synchronization. In C this is undefined behavior, not merely “unpredictable”. The compiler is allowed to assume it never happens.

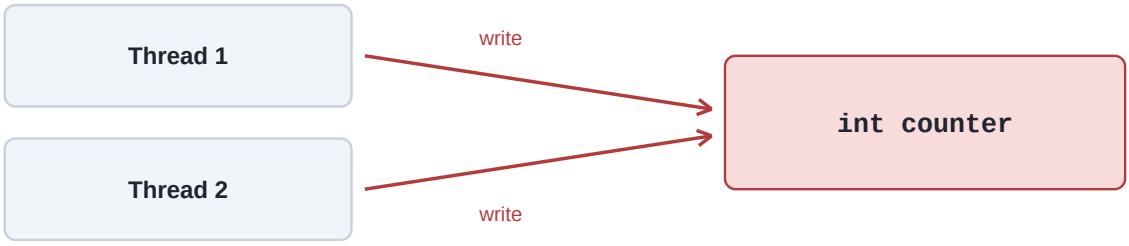

Figure 10. A data race. Two threads write the same object with no synchronization, so the result is undefined.

Resource errors.

- **Memory leak.** A block is allocated and never freed. The program keeps growing until the system runs out of memory.
- **Fragmentation.** Free memory exists, but in pieces too small to satisfy the next request. The program fails even though total free memory is sufficient.

Figure 11. Fragmentation. The holes add up to enough memory, but none of them is large enough.

Provenance and arithmetic errors.

- **Forged or out-of-object pointer.** Casting an integer to a pointer, or doing arithmetic that leaves the object, produces a pointer the standard never allowed. The machine still dereferences it.

- **Integer overflow in a size calculation.** `malloc(n * sizeof(int))` can wrap around to a tiny number when *n* is huge. The allocation succeeds, and the program then writes far past its end.

```
size_t n = get_count();
int *a = malloc(n * sizeof(int));   /* n * 4 can overflow */
a[0] = 1;                           /* small block, huge index   */
```

10 Why C is built this way

It is worth being fair to C. The design is not an accident; it is a deliberate trade, made in the early 1970s for machines with a few kilobytes of memory.

- **Trust the programmer.** C assumes you know what you are doing. Checks cost time, and in kernels and embedded systems that time matters.
- **No hidden runtime.** There is no garbage collector, no runtime that pauses your program, and no background thread moving objects. A C program does exactly what the machine instructions say.
- **Predictable performance.** Allocation is a function call, a local variable is a stack slot, and a field access is an offset. You can reason about the cost of each line.
- **Small and portable.** The language is small enough to implement on almost any chip, and close enough to the hardware to serve as a “portable assembler”.
- **Full control.** You choose the layout, the lifetime, and the aliasing. That is what lets C write operating systems, device drivers, language runtimes, and databases.

The price is that **every mistake is yours, and many mistakes are silent.** C gives you the control of assembly with the type system of a spreadsheet. That trade was reasonable when programs were small. It is much harder to justify for the multi-million-line programs we write today.

11 What people do about it

None of this means C cannot be used safely. It means safety has to come from somewhere else. The main approaches:

- **Discipline and review.** Coding standards, small functions, clear ownership rules, careful review. Cheap, but does not scale and misses subtle bugs.
- **Static analysis.** Tools that read the code and look for out-of-bounds access, leaks, and null dereferences. Catches some bugs before they run, at the cost of false positives.
- **Sanitizers.** Compiler instrumentation that checks at run time. AddressSanitizer catches out-of-bounds and use-after-free, UndefinedBehaviorSanitizer catches many UB cases, MemorySanitizer catches uninitialized reads, and ThreadSanitizer catches data races. Excellent for testing, too slow for production.
- **Hardening.** Stack canaries, address-space randomization, non-executable memory, and hardened allocators. Makes exploitation harder, but does not remove the bugs.
- **Memory tagging and capabilities.** New hardware can attach a tag or a capability to each pointer and check it on every access. Arm's MTE and CHERI-style capabilities are the leading examples. Fil-C shows that a memory-safe C can also be built in software, at a runtime cost.
- **Other languages.** Rust enforces ownership and borrowing at compile time and prevents most of the flaws in this document, at the cost of giving up some of C's freedom. Go, Java and others use garbage collection, trading control for automatic lifetime management.

12 Summary

One table, one page, every flaw in this guide. Each defense trades something: speed, memory, compatibility, or control. That trade-off is the subject of the next part of this research.

Flaw	Root cause	What goes wrong	Typical defense
Buffer overflow	no bounds checks	overwrites neighbors	bounds checks, ASan, MTE
Use-after-free	pointer outlives object	touches reused memory	ownership, GC, generations
Double free	block freed twice	corrupts the free list	null after free, hardened allocator
Memory leak	missing free	memory grows forever	review, leak checkers, RAII
Dangling stack pointer	returning a local address	points at a reused frame	return by value, escape analysis
Uninitialized read	malloc does not zero	garbage value, info leak	calloc, MSan, init tags
Type confusion	unchecked casts	bytes read as wrong type	careful casts, type tags
Data race	shared memory, no sync	undefined behavior	locks, atomics, TSan
Fragmentation	scattered free blocks	large allocation fails	pools, arenas, better allocator
Size overflow	unchecked arithmetic	too-small allocation	checked math, safe-int helpers
Forged pointer	a pointer is just a number	arbitrary access	capabilities, provenance, ASLR

C's memory model is beautifully simple: bytes, addresses, and lifetimes you manage yourself. That simplicity is the reason C is still everywhere. It is also the reason the same three bugs — overflow, use-after-free, and uninitialized memory — have been exploited for forty years. C does not have a memory-safety mechanism. It has a memory-safety **convention**, and the machine is under no obligation to enforce it. Understanding exactly where the convention ends is the first step toward designing something better.